Hochleistungsdatenhaltungssysteme

Dr. Stephan Hofmann

TH Ingolstadt

Teil 6.1: Einführung in die Anwendung von SAP HANA Graph

Überblick

Die folgende Übung gibt eine kurze Einführung in die Konzepte und Werkzeuge zur Erstellung und Analyse von Graphen innerhalb der SAP HANA DB. Hierbei werden die Werkzeuge der SAP WEBIDE und der SAP Graphviewer eingesetzt.

Einsatzszenario:

Reporting, Analyse und Interpretation von Unternehmensdaten sind von zentraler Bedeutung, um die Wettbewerbsfähigkeit von Unternehmen zu garantieren, Prozesse zu optimieren sowie marktgerecht und schnell agieren zu können. Beim Einsatz der SAP-HANA-Datenbank als Persistenz für die Daten bietet sich für die Ausführung von Analyse- und Planungsszenarien der Einsatz der integrierte Graph-Werkzeuge an. Die Modellierung und Analyse mit Hilfe von Graph-Algorithmen wird innerhalb der SAP WebIDE durchgeführt.

Inhaltsverzeichnis

Einführung in die Graphentheorie *4*

SAP HANA Graph *5*

Einordnung des Beispieles **5**

Anlegen der benötigten Tabellen **7**

Beispieldaten werden in das vorbereitete Modell importiert **9**

Anlage des graph workspace **12**

Analyse des Graphen mit Hilfe des Graph Viewer **13**

Hinzufügen weiterer Daten zum Graphenmodell **19**

Analysen auf dem Graphenmodell **19**

Einführung in die Graphentheorie

Graphen sind mathematische Modelle für netzartige Strukturen in Natur und Technik. Dazu zählen Straßennetze, Computernetze, elektrischen Schaltungen, Programmablaufpläne, Wasser und Gasleitungsnetze, chemische Moleküle oder wirtschaftliche Verflechtungsbeziehungen. Allen diesen Netzen ist eine grundlegende Eigenschaft gemein. Sie bestehen aus zwei verschiedenartigen Mengen von Objekten. Die Objekte der ersten Art sind beispielsweise Orte im Straßennetz oder Computer innerhalb eines Computernetzwerkes. Diese werden durch die zweite Art der Objekte verbunden. Das sind beispielsweise Straßen oder Überragungsleitungen.

Im Kontext der Graphentheorie werden diese Objekte als Knoten und Kanten eines Graphen bezeichnet. Viele Anwendungen der Graphentheorie erfordern die Untersuchung der speziellen Eigenschaften des jeweiligen Netzes.

Für die Planung einer Reise ist die Bestimmung des kürzesten Weges zwischen 2 Orten eines Straßennetzes ein interessantes Problem.

Zielsetzung:

In dieser Übung wird ein grober Überblick über die Möglichkeiten von SAP HANA Graph gegeben. Darauf aufbauend werden einige Beispieldaten in das Graphmodell geladen und ein SAP HANA Graph Workspace erzeugt, der dann im Laufe der Übung analysiert wird.

Nach dem Abschluss der Übung sollten folgende Inhalte bekannt und verstanden sein:

- Was ist SAP HANA Graph
- Wie legt man mit Hilfe von Beispieldaten einen Graph Workspace an
- Welche Möglichkeiten der Strukturanalyse werden genutzt

SAP HANA Graph

SAP HANA Graph ist ein integraler Bestandteil der SAP HANA Kernfunktionalität. Es erweitert die Funktionen der SAP HANA um einen nativen Support für graph-basierte Analysen und Prozesse und ermöglicht es auf einfache Weise typische Graphen Operationen direkt auf Daten auszuführen, die innerhalb der SAP HANA abgelegt sind.

In SAP HANA, wird ein Graph durch eine Menge an Knoten (vertex/node) und eine Menge an Kanten (edge) repräsentiert. Jede Kante verbindet hierbei genau zwei Knoten miteinander. Ein Knoten wird hierbei als Quellknoten betrachtet und der zweite Knoten als Ziel Kanten werden immer mit einer Richtung versehen und verbinden zwei Knoten miteinander. Es können jedoch auch mehrere Kanten modelliert werden, die dieselben Knoten miteinander verbinden. Knoten und Kanten können zusätzlich eine beliebige Anzahl an Attributen haben. Ein Knotenattribut besteht jeweils aus einem Namen, die mit einem definierten Datentyp verknüpft ist und einem Wert. Kantenattribute bestehen aus den gleichen Informationen. Nachfolgend ist ein einfaches Beispiel dargestellt.

Einordnung des Beispieles

Zur Modellierung und als Datenbasis nutzen wir einen Teil eines Ski-Resort-Gebietes in Italien (Pinzolo/Madonna di Campiglio). Ein Ausschnitt des Gebietes ist nachfolgend dargestellt.

Jede Station besitzt eine eindeutige Nummer i.S. einer ID. Diese sind als rote Kreise auf der Zeichnung gekennzeichnet und werden als Knoten in unserem Graphmodell abgebildet.

Zusätzlich zur ID werden den Knoten noch weitere Attribute zugeordnet:

- `ticket_office = 'TRUE'` (Datentyp string, not boolean), Hier können die Tickets gekauft werden und die Tour beginnt an diesen Punkten. Die Stationen 1, 4, 15 sind die Stationen mit diese Eigenschaft.

- `restaurant = 'TRUE'` ist selbstsprechend. Hier kann man in einem Restaurant Essen gehen. Nur an den Stationen 2, 3, 5 wird diese Möglichkeit geboten.

Alle Stationen sind durch Lifts miteinander verbunden, um nach oben zu gelangen oder um ins Tal zu fahren. Diese Verbindungen werden im Graphmodell als Kanten abgebildet und enthalten neben der ID ebenfalls weitere Eigenschaften.

Über den Modus (mode == ‘lift’ oder ‘run’) wird die Richtung angegeben

- `start` und `end` sind die jeweiligen IDs der miteinander verbundenen Stationen.

- Jede Verbindung wir mit dem Attribut `length` versehen, das die Länge der Verbindung in Meter angibt,

- Um den Schwierigkeitsgrad der jeweiligen Route zu spezifizieren, werden die Werte ‘blue’, ‘red’, oder ‘black’ für das Attribut `difficulty` verwendet.

- Das Attribut `status` kann entweder ‘open’ oder ‘close’ sein. Hierbei wird definiert ob der jeweilige Lift in Betrieb ist und genutzt werden kann. Als Standardfall geht man davon aus, dass alle Verbindungen offen sind.

Anlegen der benötigten Tabellen

Zunächst öffnet man den SAP HANA SQL Editor um sich mit der Datenbank zu verbinden. Für die anzulegenden Strukturen wird das jeweils eigene Schema verwendet. Mit Hilfe der nachfolgenden SQL-Anweisungen werden im ersten Schritt die notwendigen Tabellen angelegt.

Bitte beachten Sie die zugrundeliegende Namenskonventionen!

```
CREATE TABLE "GBI_DEMO_DATA"."Dxxx_NODES"(

     "node_id" INTEGER NOT NULL,
     "name" NVARCHAR(16),
     "ticket_office" NVARCHAR(4),
     "restaurant" NVARCHAR(4),
     PRIMARY KEY (
          "node_id"
     )
);

CREATE TABLE "GBI_DEMO_DATA"."D000_EDGES"(

     "edge_id" INTEGER NOT NULL,

     "length" INTEGER,

     "difficulty" NVARCHAR(16),

     "start" INTEGER NOT NULL,

     "end" INTEGER NOT NULL,

     "mode" NVARCHAR(8),

     "status" NVARCHAR(8),

     PRIMARY KEY (

          "edge_id"
```

```
    )
);
```

Mit Hilfe der nachfolgenden Statements wird sichergestellt, dass das Graphenmodell auch in den beiden Tabellen aktuell abgebildet ist und es keine Kanten gibt, denen keine existierenden Knoten zugeordnet werden.

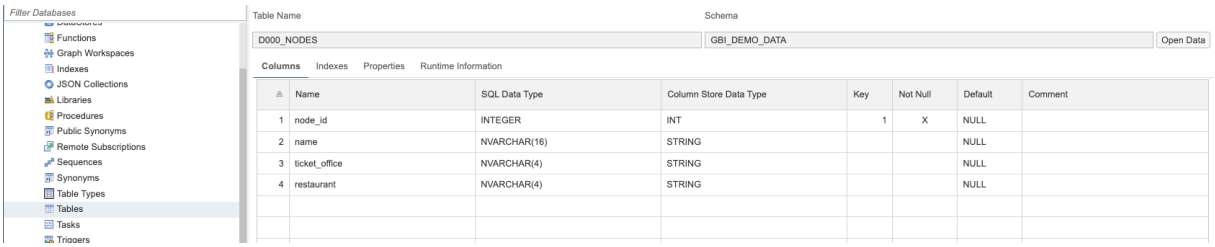
```
ALTER TABLE "GBI_DEMO_DATA"."Dxx_EDGES" ADD FOREIGN KEY
("start") REFERENCES "GBI_DEMO_DATA"."D0xx_NODES" ("node_id")
ON UPDATE CASCADE ON DELETE CASCADE;
```

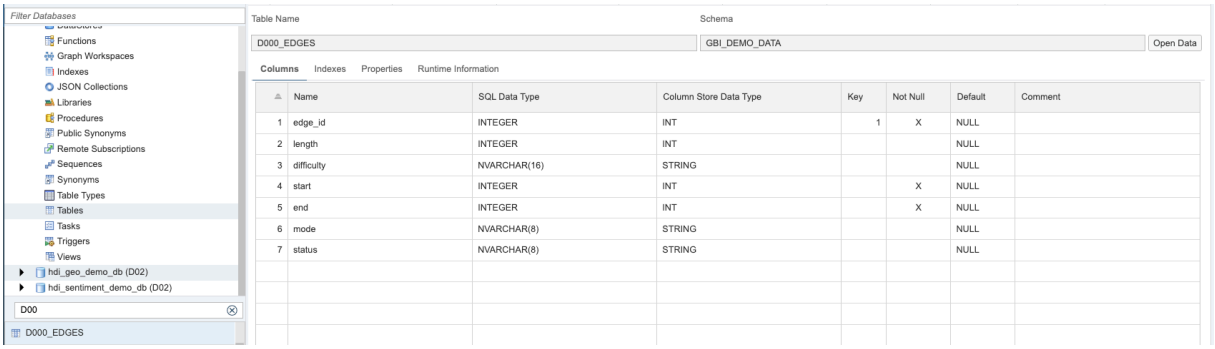
```
ALTER TABLE "GBI_DEMO_DATA"."D0xx_EDGES" ADD FOREIGN KEY (
"end" ) REFERENCES "GBI_DEMO_DATA"."D0xx_NODES" ("node_id") ON
UPDATE CASCADE ON DELETE CASCADE;
```

Als Ergebnis sind die beiden Tabellen im eigenen Schema sichtbar. Im Katalog des eigenen Schemas werden die beiden Tabellen (nach einem Refresh) angezeigt.

Mit Hilfe des Kontextmenüs können die neu angelegten Tabellen angeschaut und die inhaltliche Korrektheit geprüft werden.

Nachfolgend sind die beiden Tabellen im Data-Definition View dargestellt.





Beispieldaten werden in das vorbereitete Modell importiert

Führen Sie die nachfolgenden Anweisungen aus, um die Beispieldaten in die vorbereiteten Tabellen zu laden.

Ersetzen Sie die entsprechenden Tabellennamen durch die von ihnen angelegten Tabellen.

NODES → D0xx_NODES

EDGES → D0xx_EDGES

```
--Populate NODES
INSERT INTO "NODES" VALUES (1, 'Pinzolo', 'TRUE', '') ;
INSERT INTO "NODES" VALUES (2, 'Pra Rodont', '', 'TRUE') ;
INSERT INTO "NODES" VALUES (3, 'Dos Del Sabion', '', 'TRUE') ;
INSERT INTO "NODES" VALUES (4, 'Tulot', 'TRUE', '') ;
INSERT INTO "NODES" VALUES (5, 'Malga Ciocia', '', 'TRUE') ;
INSERT INTO "NODES" VALUES (6, 'Malga Ciocia Up', '', '') ;
INSERT INTO "NODES" VALUES (7, 'Zapel', '', '') ;
INSERT INTO "NODES" VALUES (8, 'Zapel Down', '', '') ;
INSERT INTO "NODES" VALUES (9, 'Grual', '', '') ;
INSERT INTO "NODES" VALUES (10, 'Mte Grual', '', '') ;
```

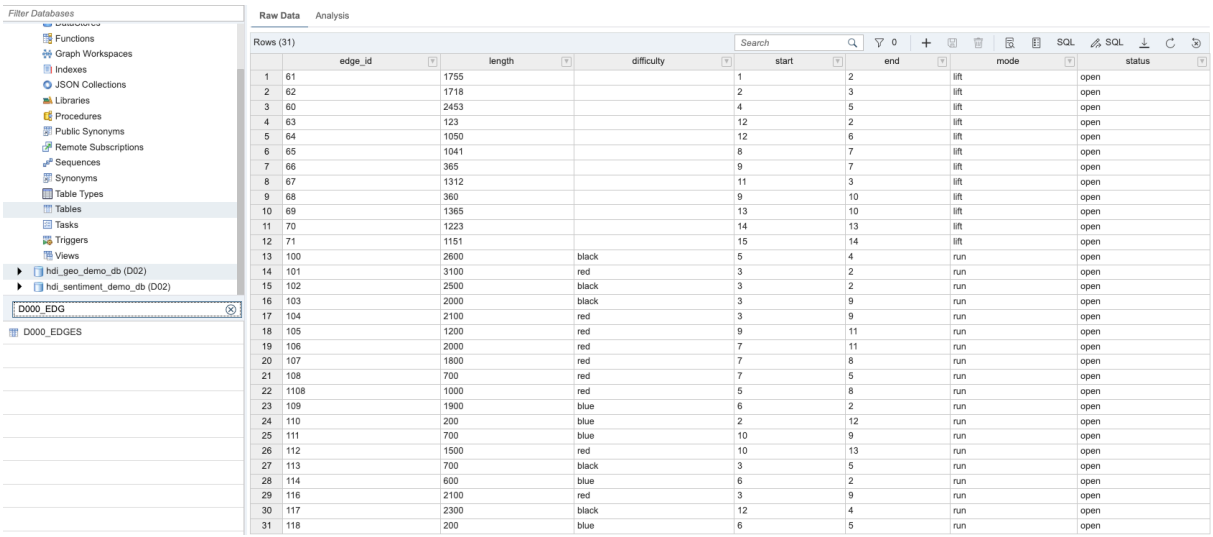
```sql
INSERT INTO "NODES" VALUES (11, 'Malga Grual', '', '') ;
INSERT INTO "NODES" VALUES (12, 'Fossadei', '', '') ;
INSERT INTO "NODES" VALUES (13, 'Puza Daifo', '', '') ;
INSERT INTO "NODES" VALUES (14, 'Plaza', '', '') ;
INSERT INTO "NODES" VALUES (15, 'Colarin', 'TRUE', '') ;

--Populate EDGES
INSERT INTO "EDGES" VALUES (61, 1755, '', 1, 2, 'lift',
'open') ;
INSERT INTO "EDGES" VALUES (62, 1718, '', 2, 3, 'lift',
'open') ;
INSERT INTO "EDGES" VALUES (60, 2453, '', 4, 5, 'lift',
'open') ;
INSERT INTO "EDGES" VALUES (63, 123, '', 12, 2, 'lift',
'open') ;
INSERT INTO "EDGES" VALUES (64, 1050, '', 12, 6, 'lift',
'open') ;
INSERT INTO "EDGES" VALUES (65, 1041, '', 8, 7, 'lift',
'open') ;
INSERT INTO "EDGES" VALUES (66, 365, '', 9, 7, 'lift', 'open')
;
INSERT INTO "EDGES" VALUES (67, 1312, '', 11, 3, 'lift',
'open') ;
INSERT INTO "EDGES" VALUES (68, 360, '', 9, 10, 'lift',
'open') ;
INSERT INTO "EDGES" VALUES (69, 1365, '', 13, 10, 'lift',
'open') ;
INSERT INTO "EDGES" VALUES (70, 1223, '', 14, 13, 'lift',
'open') ;
INSERT INTO "EDGES" VALUES (71, 1151, '', 15, 14, 'lift',
'open') ;
INSERT INTO "EDGES" VALUES (100, 2600, 'black', 5, 4, 'run',
'open') ;
INSERT INTO "EDGES" VALUES (101, 3100, 'red', 3, 2, 'run',
'open') ;
INSERT INTO "EDGES" VALUES (102, 2500, 'black', 3, 2, 'run',
'open') ;
INSERT INTO "EDGES" VALUES (103, 2000, 'black', 3, 9, 'run',
'open') ;
INSERT INTO "EDGES" VALUES (104, 2100, 'red', 3, 9, 'run',
'open') ;
INSERT INTO "EDGES" VALUES (105, 1200, 'red', 9, 11, 'run',
'open') ;
INSERT INTO "EDGES" VALUES (106, 2000, 'red', 7, 11, 'run',
'open') ;
INSERT INTO "EDGES" VALUES (107, 1800, 'red', 7, 8, 'run',
'open') ;
INSERT INTO "EDGES" VALUES (108, 700, 'red', 7, 5, 'run',
'open') ;
INSERT INTO "EDGES" VALUES (1108, 1000, 'red', 5, 8, 'run',
'open') ;
```

```sql
INSERT INTO "EDGES" VALUES (109, 1900, 'blue', 6, 2, 'run',
'open') ;
INSERT INTO "EDGES" VALUES (110, 200, 'blue', 2, 12, 'run',
'open') ;
INSERT INTO "EDGES" VALUES (111, 700, 'blue', 10, 9, 'run',
'open') ;
INSERT INTO "EDGES" VALUES (112, 1500, 'red', 10, 13, 'run',
'open') ;
INSERT INTO "EDGES" VALUES (113, 700, 'black', 3, 5, 'run',
'open') ;
INSERT INTO "EDGES" VALUES (114, 600, 'blue', 6, 2, 'run',
'open') ;
INSERT INTO "EDGES" VALUES (116, 2100, 'red', 3, 9, 'run',
'open') ;
INSERT INTO "EDGES" VALUES (117, 2300, 'black', 12, 4, 'run',
'open') ;
INSERT INTO "EDGES" VALUES (118, 200, 'blue', 6, 5, 'run',
'open') ;
```

Mit Hilfe der Data Preview Funktion der SAP WEBIDE werden die importierten Daten in den Tabellen angezeigt.

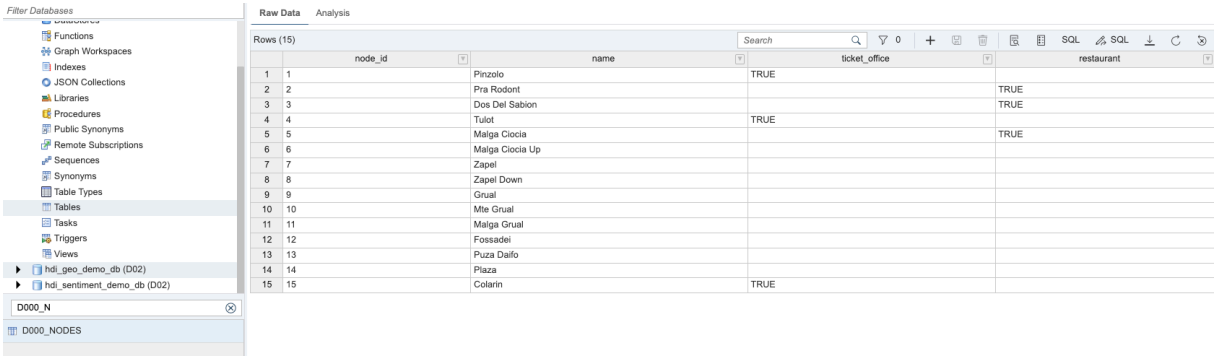
Hierzu wechselt man auf eine der beiden Tabellen und öffnet mit Hilfe des Kontext-Menüs den Data Preview Bereich.

Zunächst die Tabelle „D0xx_EDGES“.



	edge_id	length	difficulty	start	end	mode	status
1	61	1755		1	2	lift	open
2	62	1718		2	3	lift	open
3	60	2453		4	5	lift	open
4	63	123		12	2	lift	open
5	64	1050		12	6	lift	open
6	65	1041		8	7	lift	open
7	66	365		9	7	lift	open
8	67	1312		11	3	lift	open
9	68	360		9	10	lift	open
10	69	1365		13	10	lift	open
11	70	1223		14	13	lift	open
12	71	1151		15	14	lift	open
13	100	2600	black	5	4	run	open
14	101	3100	red	3	2	run	open
15	102	2500	black	3	2	run	open
16	103	2000	black	3	9	run	open
17	104	2100	red	3	9	run	open
18	105	1200	red	9	11	run	open
19	106	2000	red	7	11	run	open
20	107	1800	red	7	8	run	open
21	108	700	red	7	5	run	open
22	1108	1000	red	5	8	run	open
23	109	1900	blue	6	2	run	open
24	110	200	blue	2	12	run	open
25	111	700	blue	10	9	run	open
26	112	1500	red	10	13	run	open
27	113	700	black	3	5	run	open
28	114	600	blue	6	2	run	open
29	116	2100	red	3	9	run	open
30	117	2300	black	12	4	run	open
31	118	200	blue	6	5	run	open

Darauffolgend die Tabelle „NODES“.



	node_id	name	ticket_office	restaurant
1	1	Pinzolo	TRUE	
2	2	Pra Rodont		TRUE
3	3	Dos Del Sabion		TRUE
4	4	Tulot	TRUE	
5	5	Malga Ciocia		TRUE
6	6	Malga Ciocia Up		
7	7	Zapel		
8	8	Zapel Down		
9	9	Grual		
10	10	Mte Grual		
11	11	Malga Grual		
12	12	Fossadei		
13	13	Puza Daifo		
14	14	Plaza		
15	15	Colarin	TRUE	

Als Ergebnis sollten 15 Zeilen in der Tabelle `NODES` und 31 Zeilen in der Tabelle `EDGE` angezeigt werden.

Anlage des graph workspace

Mit Hilfe eines einfachen SQL-Befehls wird aus den beiden angelegten Tabellen ein Graphenmodell erzeugt und im Graphworkspace abgespeichert.

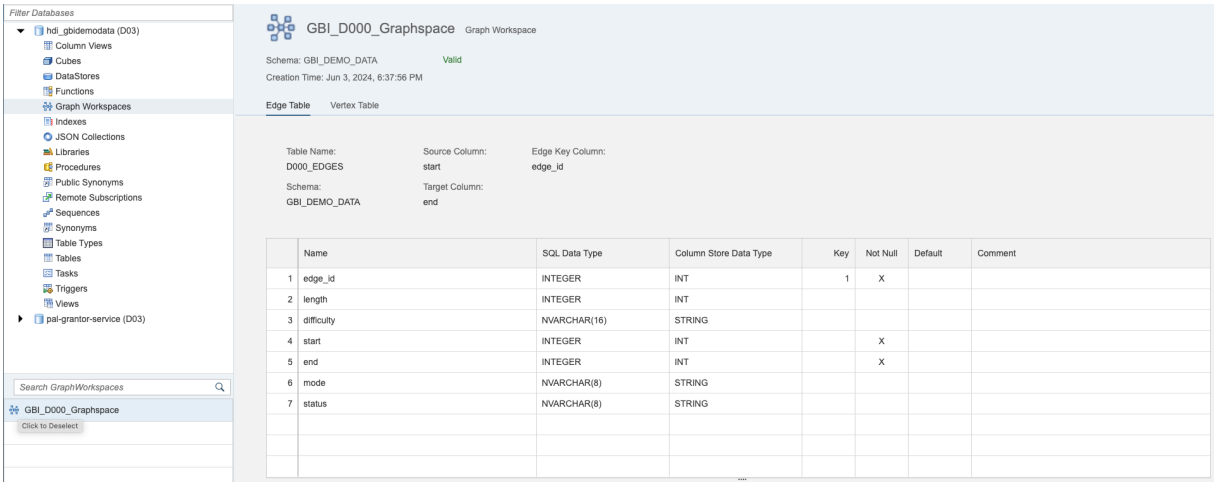
Um die Workspaces eindeutig zu identifizieren, muss ein entsprechend aussagekräftiger Namen genutzt werden. Hierbei ist es hilfreich das eigene Schema als Bestandteil des Workspacenamens zu verwenden und die Modellbezeichnung als zusätzliche Kennung.

Für das Schema „GBI_000“ wird die folgende Definition verwendet.

```
CREATE GRAPH WORKSPACE "GBI_D0xx_Graphspace"
  EDGE TABLE "GBI_DEMO_DATA"."D0xx_EDGES"
    SOURCE COLUMN "start"
    TARGET COLUMN "end"
    KEY COLUMN "edge_id"
  VERTEX TABLE "GBI_DEMO_DATA"."D0xx_NODES"
    KEY COLUMN "node_id"
;
```

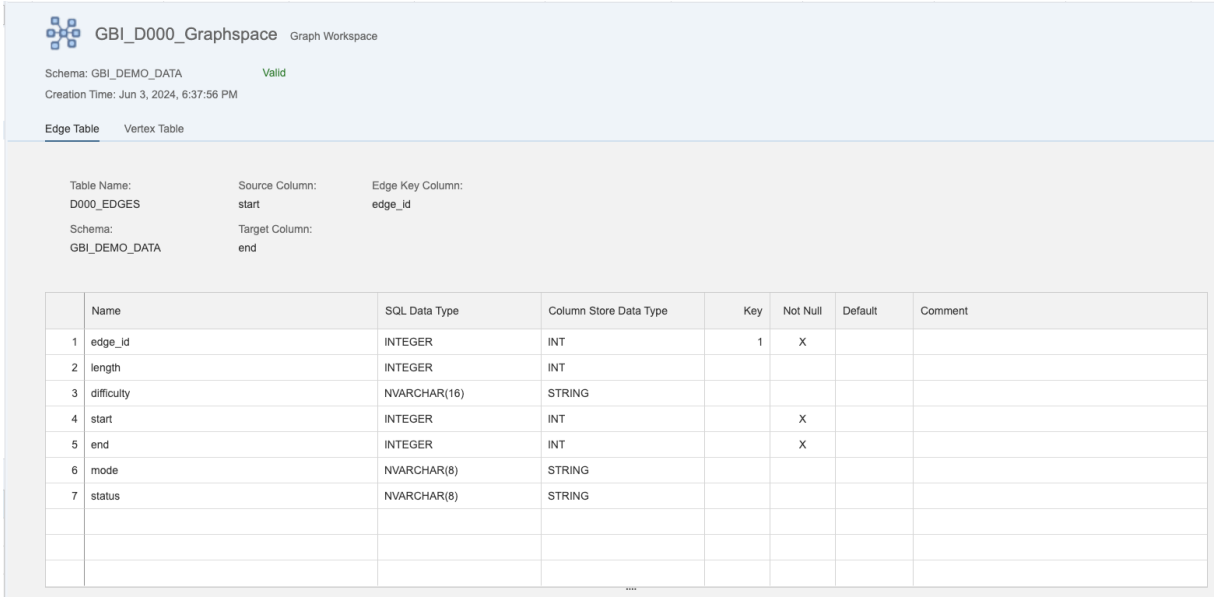
Analyse des Graphen mit Hilfe des Graph Viewer

Nachdem der notwendige Graphspace angelegt wurde, erscheint dieser ebenfalls in der Anzeige.

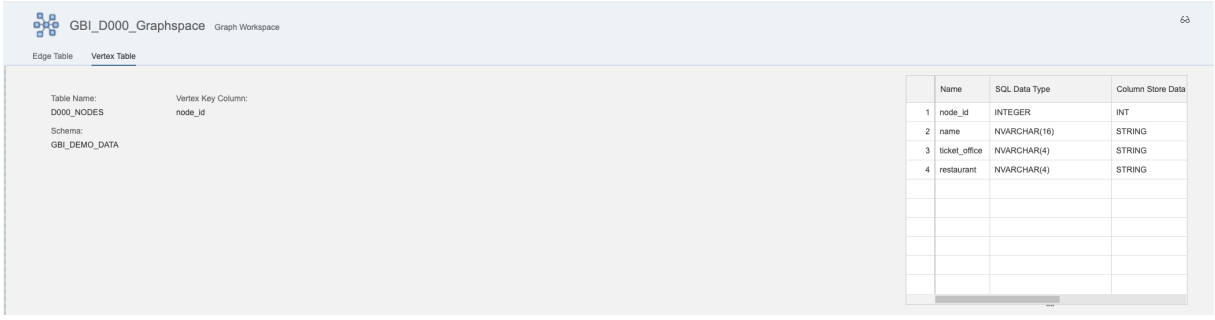


Mit Hilfe der integrierten Werkzeuge kann der erzeugte Graph analysiert werden.

Die Implementierung der Kanten wird in der Edge Table angezeigt.

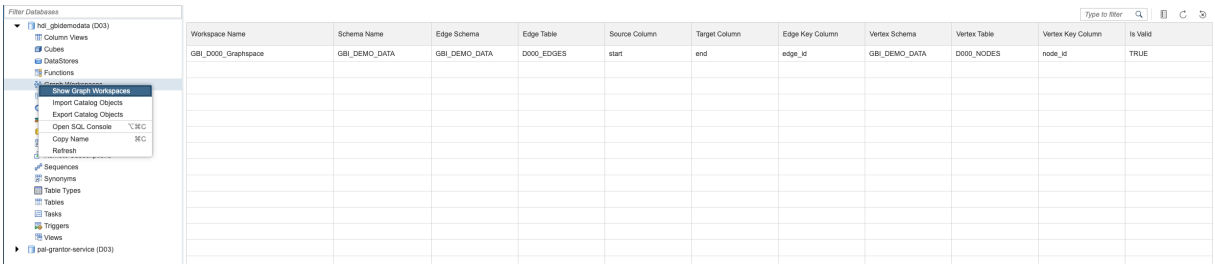


Mit dem Wechsel auf die Vertex Table wird die Knotenimplementierung angezeigt.

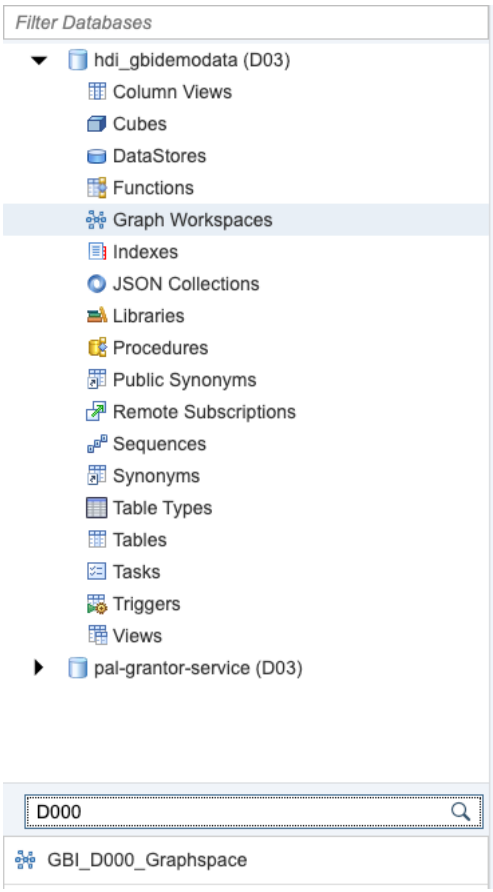


In der Auswahlliste können alle existierenden Graphenworkspaces ausgewählt werden.

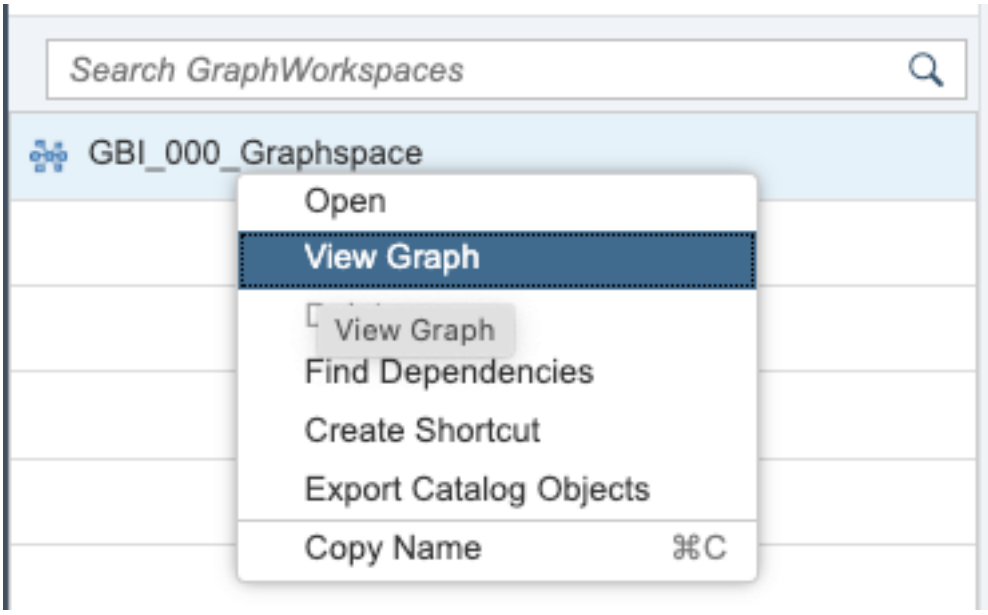
Die Übersicht über alle existierenden Graph-Workspaces ist über das Kontext-Menü erreichbar. Die verwendeten Strukturen zur Definition des Graphen sind hier dargestellt.



Um den eigenen Workspace identifizieren zu können, ist die vorgestellte Namenskonvention wichtig.

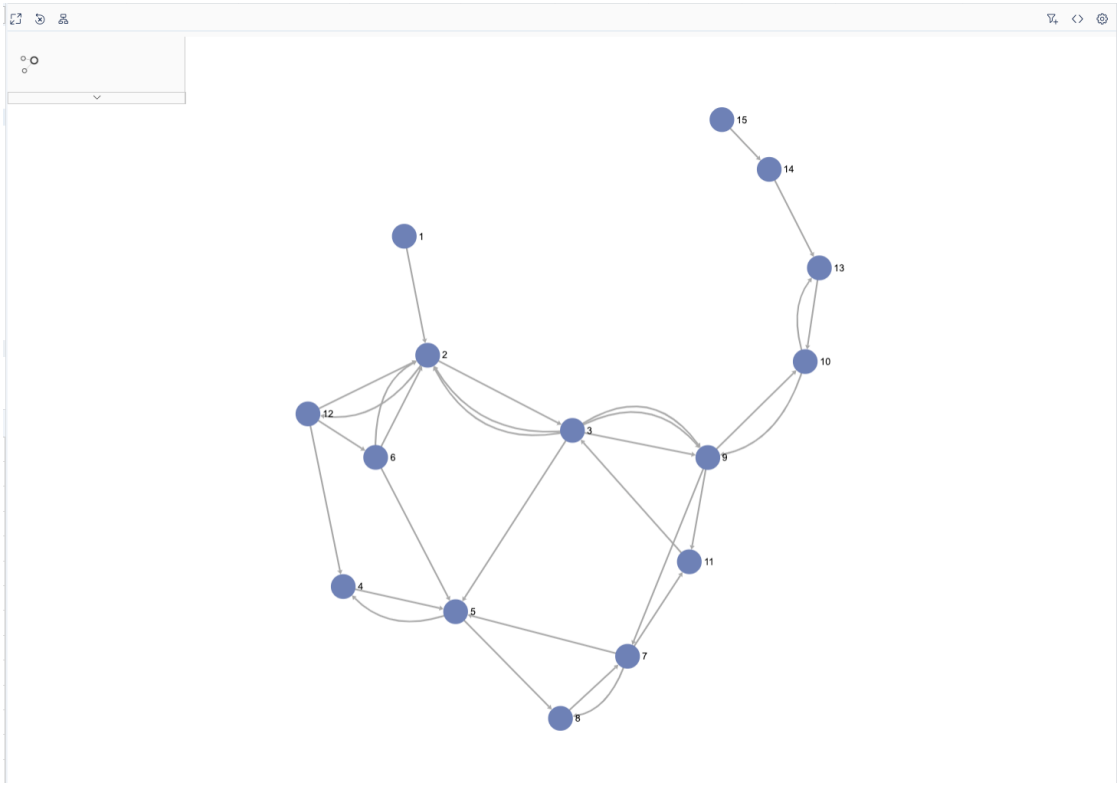


Zur Analyse des Graphen sind verschiedene Wege möglich. Mit dem integrierten Kontextmenü kann die Anzeigeanwendung des ausgewählten Graphen gestartet werden.

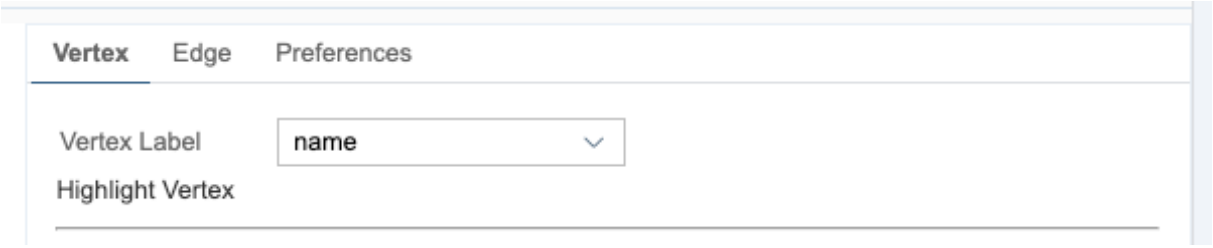


Eine weitere Möglichkeit ist die Anzeigeoption im oberen rechten Bereich des Graphworkspaces.

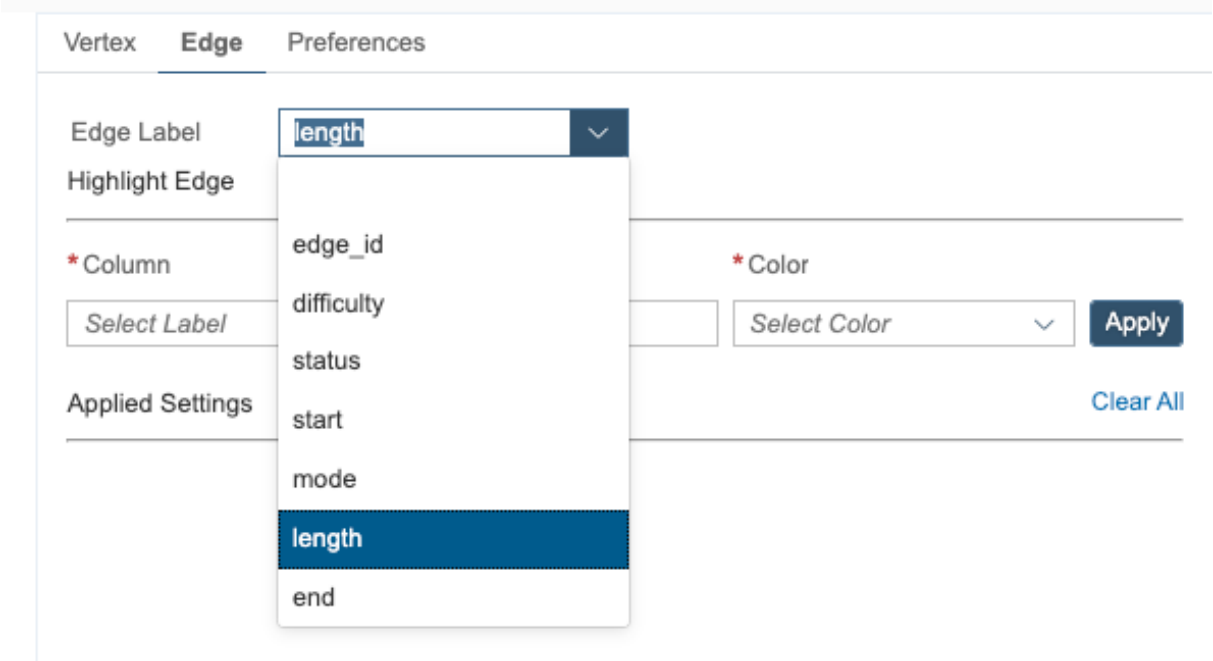
Nach der Auswahl des Workspaces wird der Graph vollständig angezeigt.



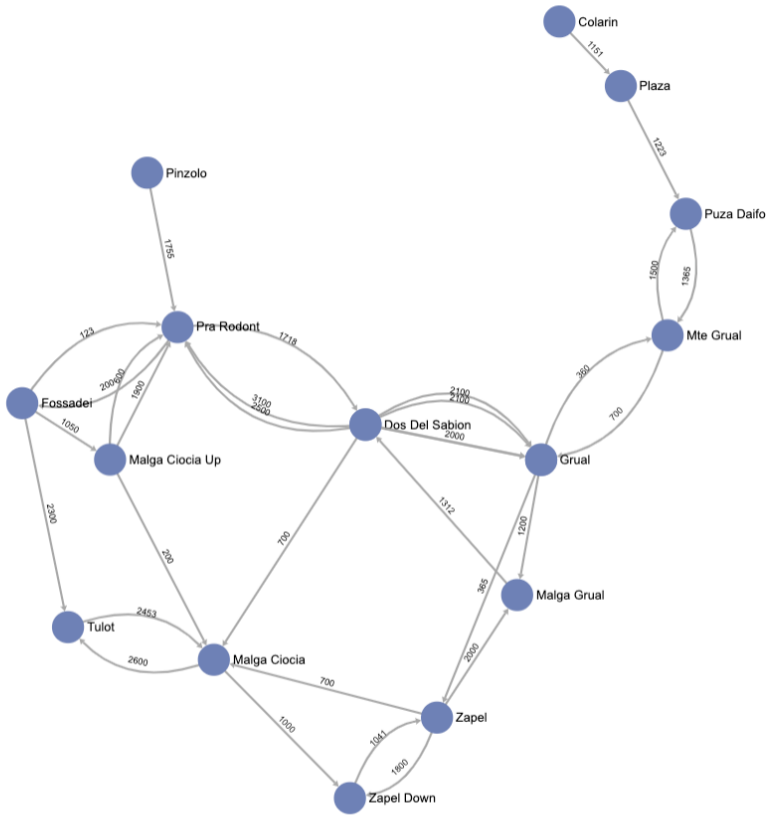
Über die Option „Settings“ im oberen Bereich können die Anzeigeoptionen für den Graphen definiert werden. Beispielsweise können die Knoten-Label zur Anzeige eingestellt werden.



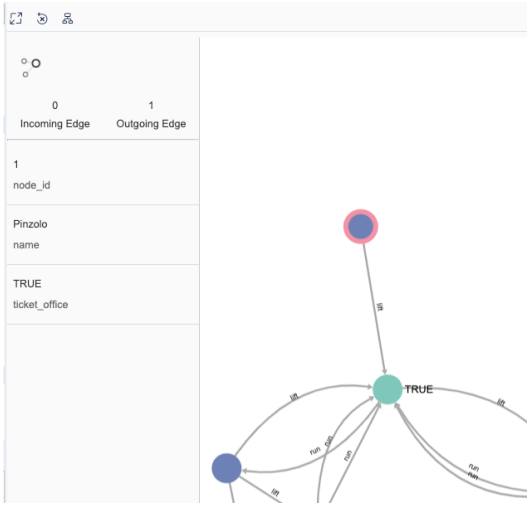
Vergleichbare Optionen stehen auch für die Kantenrepräsentation zur Verfügung.

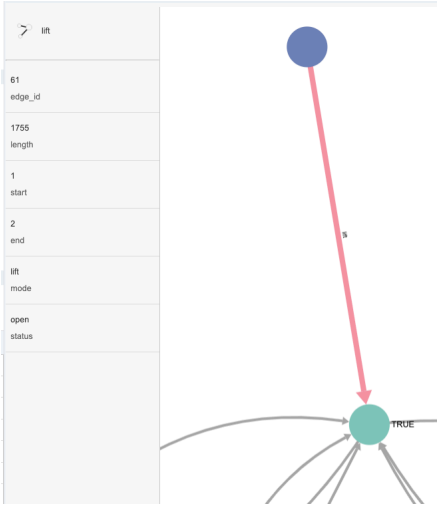


Damit erhöht sich der Informationsgehalt der Darstellung.



Durch das Selektieren eines Knotens oder einer Kante können relevante Informationen zum entsprechenden Element zur Anzeige gebracht werden.





Hinzufügen weiterer Daten zum Graphenmodell

In der Realität starten die Lift immer vom Stationen mit einem Ticket-Store. Diese können genauso auch ein Endpunkt einer Route sein. Somit kann diese Tatsache mit Hilfe einiger weiterer Kanten im Graphenmodell abgebildet werden.

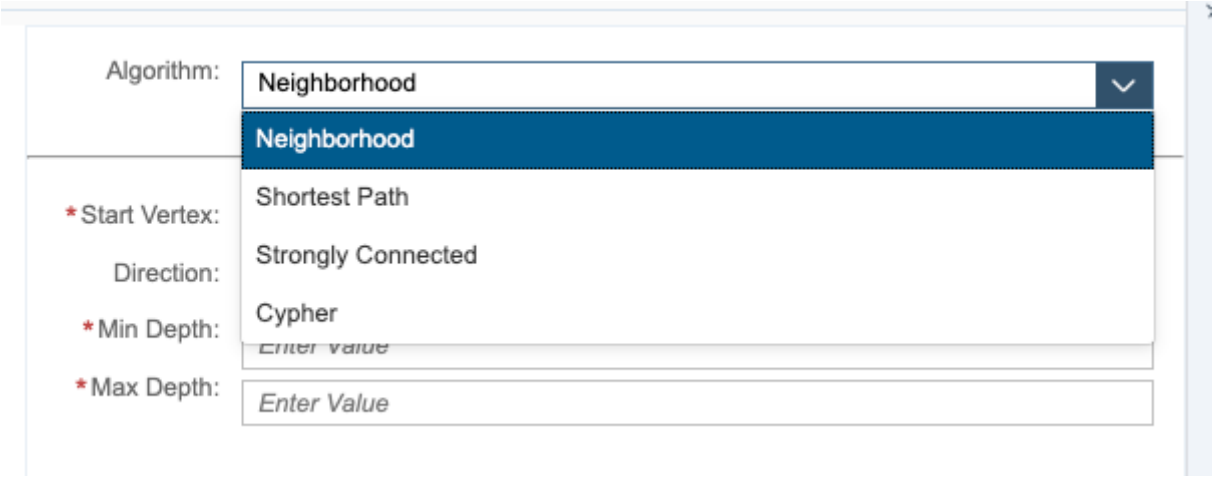
Wir fügen weitere Datensätze zum Graphenmodell hinzu. Dafür nutzen wir die nachfolgenden SQL Anweisungen.

```
--Add lifts down to boarding stations

INSERT INTO "GBI_DEMO_DATA"."D000_EDGES" VALUES (1061, 1755, '',
2, 1, 'lift', 'open');

INSERT INTO "GBI_DEMO_DATA"."D000_EDGES" VALUES (1060, 2453, '',
5, 4, 'lift', 'open');

INSERT INTO "GBI_DEMO_DATA"."D000_EDGES" VALUES (1070, 1223, '',
13, 14, 'lift', 'open');

INSERT INTO "GBI_DEMO_DATA"."D000_EDGES" VALUES (1071, 1151, '',
14, 15, 'lift', 'open');
```

Analysen auf dem Graphenmodell

Für die Strukturanalyse des Graphmodells sind verschiedene Algorithmen verfügbar:

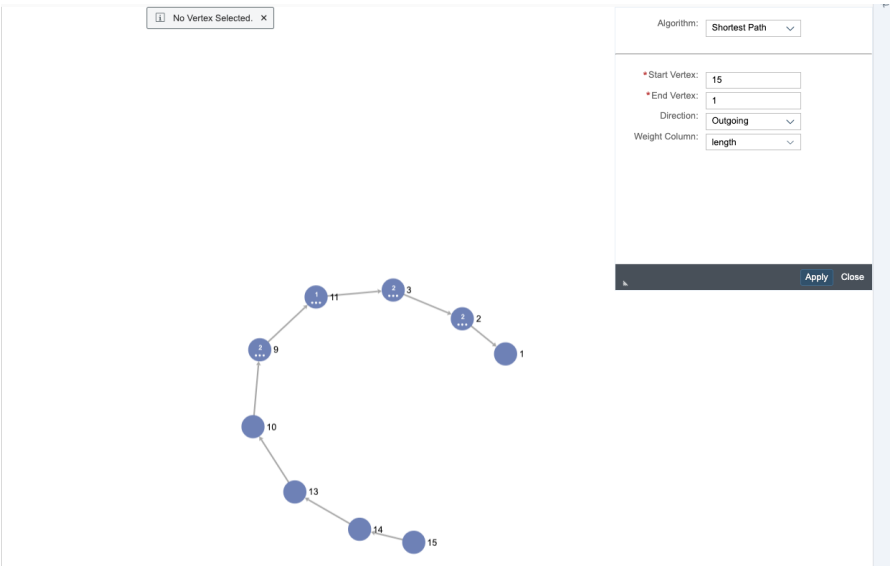
Die Auswahl ist über das Menü verfügbar.



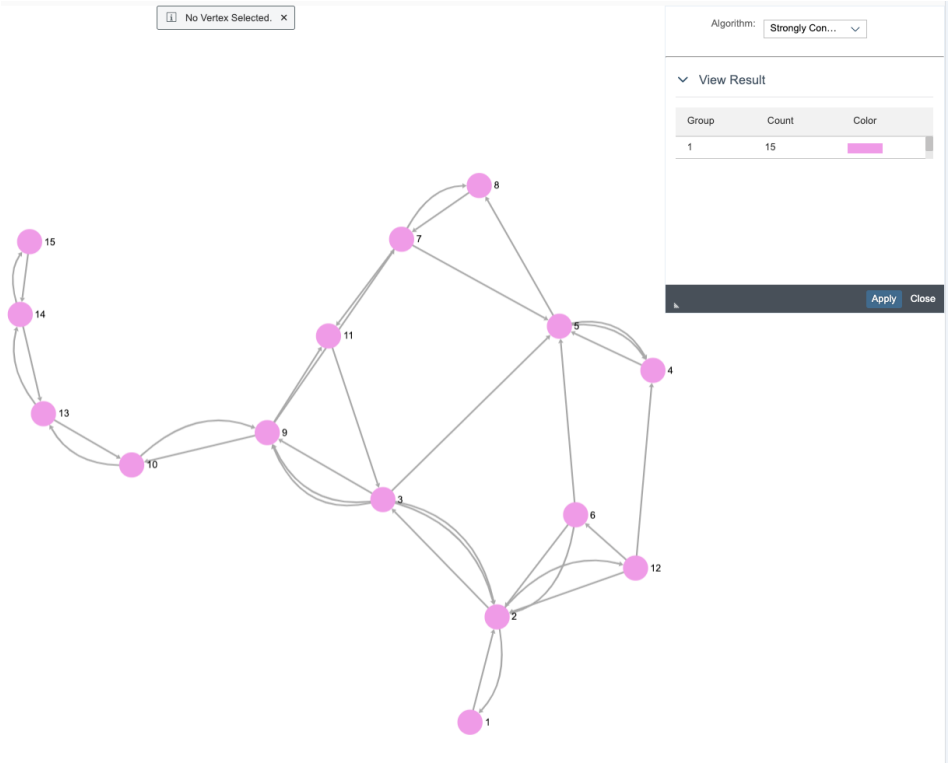
Anwendung:

1. Auswahl des Algorithmus
2. Definition der Eingabeparameter
3. „Apply“ zur Ausführung des jeweiligen Algorithmus und darstellung im visuellen Graphenmodell

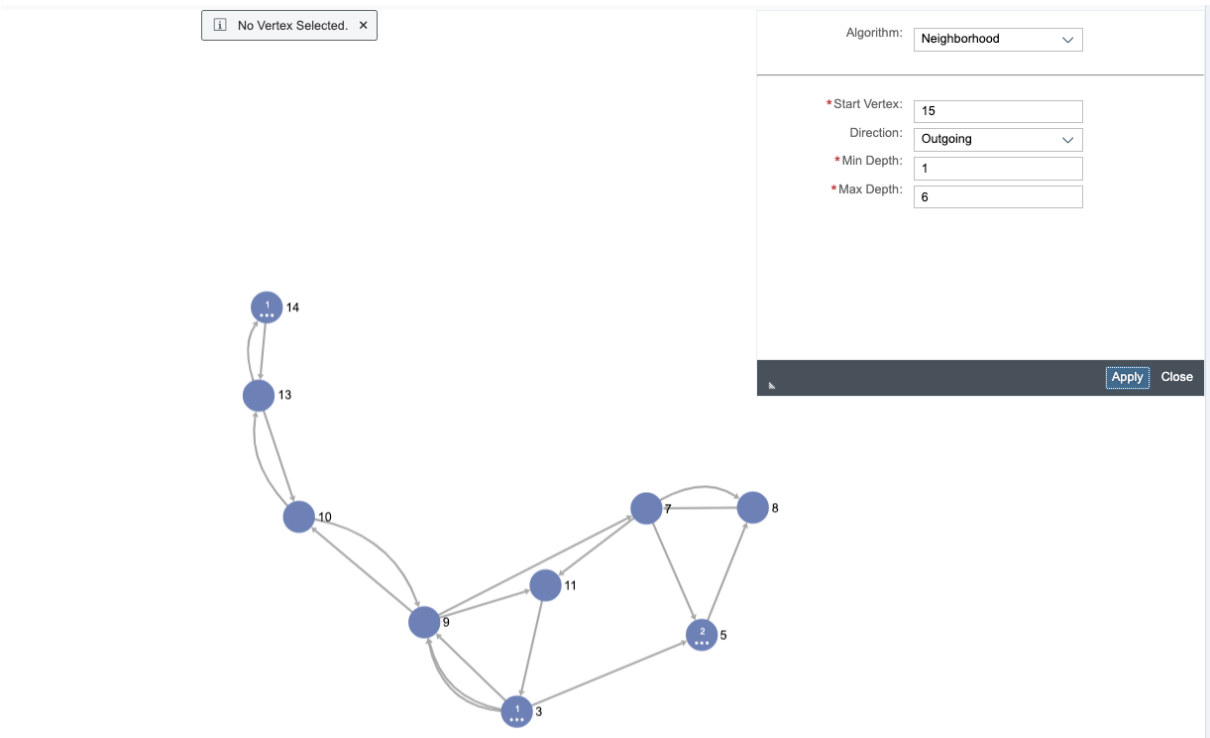
Shortest Path:



Strongly Connected:



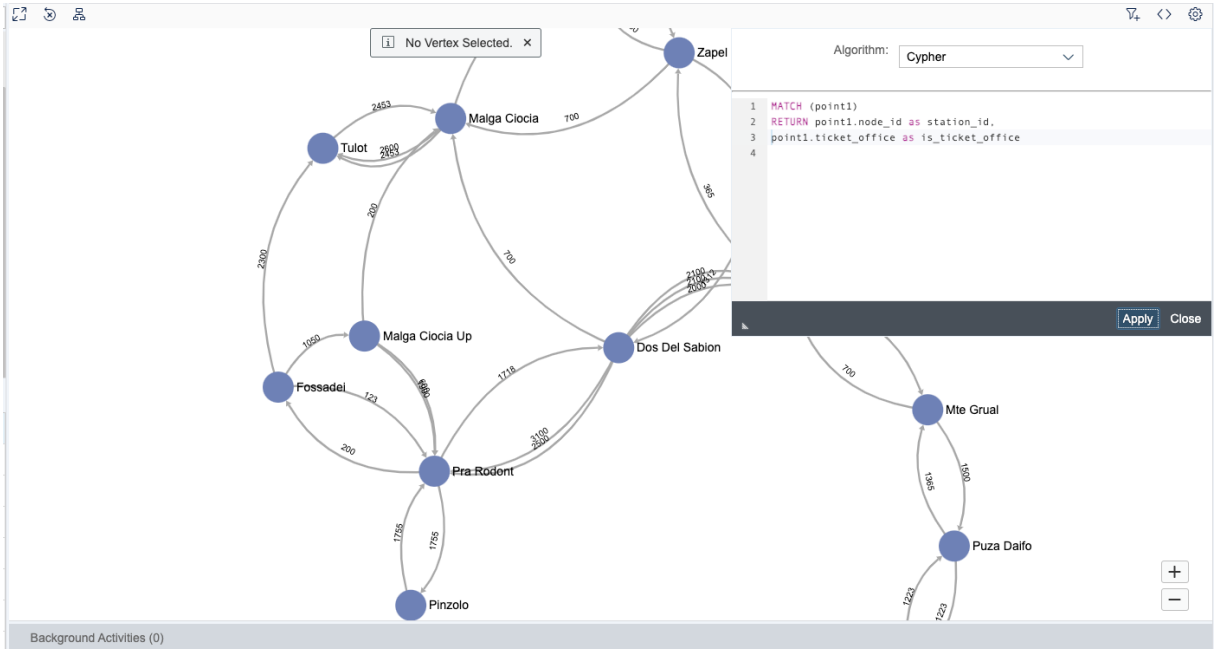
Neighborhood:



(Open)Cypher:

OpenCypher ist eine deklarative Graphabfragesprache um die Abfrage von Graph Pattern Matching zu implementieren. Diese Implementierung steht in SAP HANA Graph zur Verfügung. Sie wurde von der openCypher Implementers Group entwickelt.

Mit Hilfe der integrierten Cypher Implementierung können auch einfache Pattern-Matching Abfragen durchgeführt werden. Die Abfragesprache unterscheidet sich von der cypher-Implementierung innerhalb Neo4J.



Im entsprechenden Feld kann die Cypherabfrage eingegeben werden und via „Apply“ wird der passende Sub-Graph angezeigt. Mit Hilfe eines iterativen Vorgehens kann die jeweilige Abfrage sukzessive weiterentwickelt werden.

Nutzen Sie die folgenden Cypher-Abfragen, und interpretieren Sie die jeweiligen Ergebnisse:

```
MATCH (point1)
RETURN point1.node_id as station_id, point1.ticket_office as is_ticket_office
```

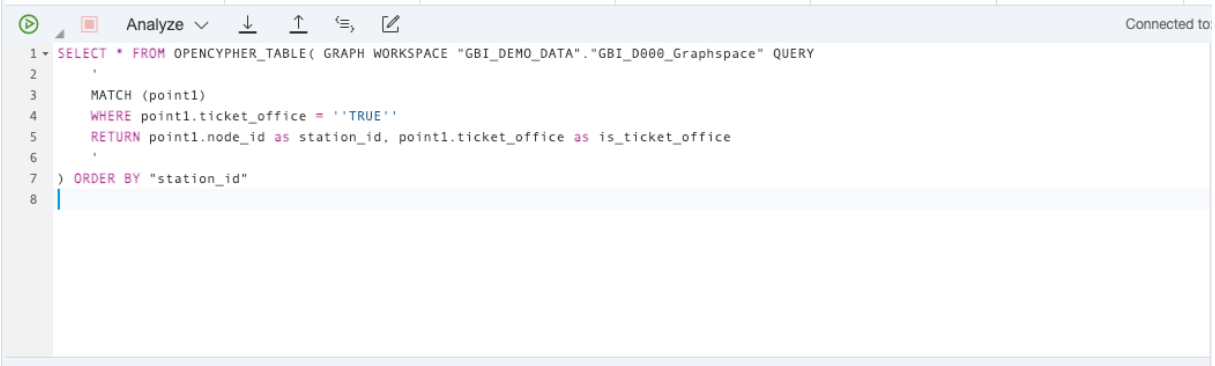
```
MATCH (point1)  WHERE point1.ticket_office = 'TRUE'
RETURN point1.node_id as station_id, point1.ticket_office as is_ticket_office
```

```
MATCH (point1)  WHERE point1.restaurant = 'TRUE'
RETURN point1.node_id as station_id, point1.restaurant as is_restaurant, point1.name as name_of_office
```

Der entsprechende Graph wird innerhalb einer dedizierten SAP HANA Tabelle, „OPENCYPHER_TABLE“ abgebildet.

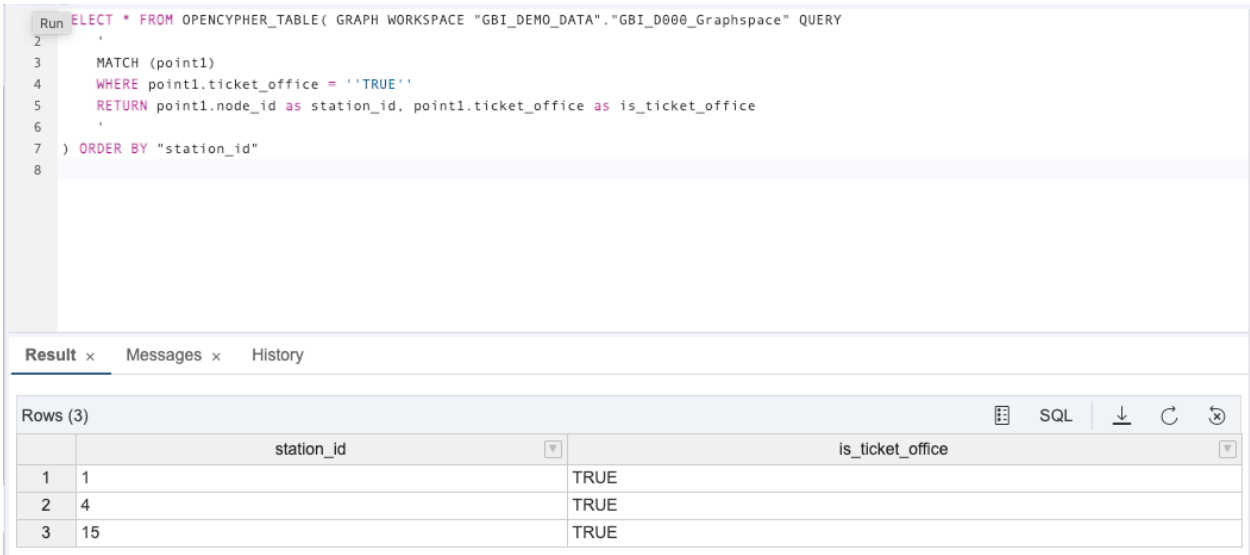
Somit kann die entsprechende Abfrage des aufgebauten Graph-Index auch mit Hilfe eines adäquaten SQL-Statements durchgeführt werden.

Hier ein Beispiel:



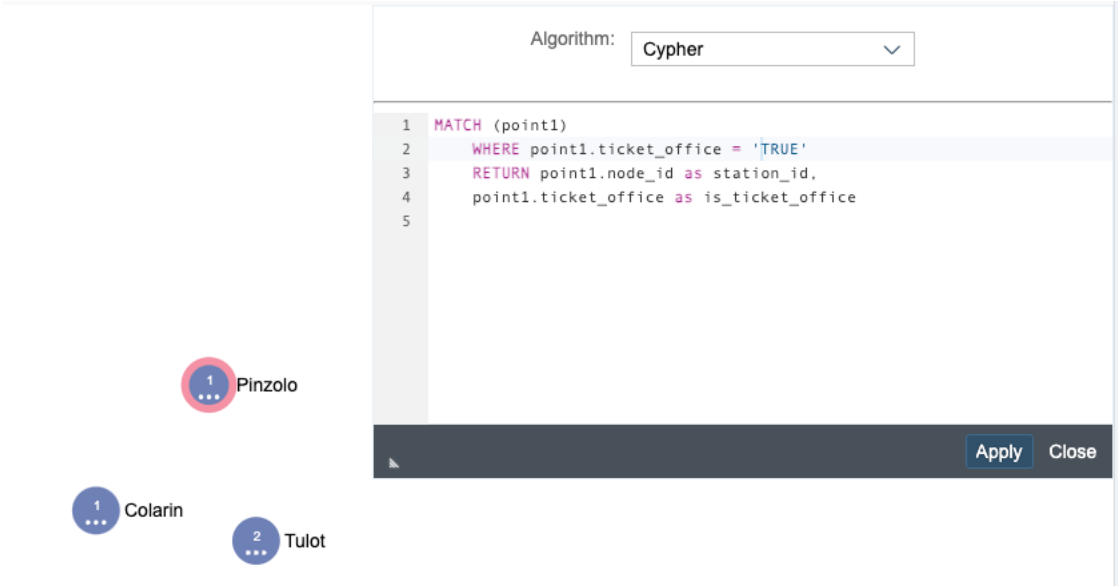

```
SELECT * FROM OPENCYPHER_TABLE( GRAPH WORKSPACE
"GBI_DEMO_DATA"."GBI_D000_Graphspace" QUERY
  '
  MATCH (point1)
  WHERE point1.ticket_office = ''TRUE''
  RETURN point1.node_id as station_id, point1.ticket_office as is_ticket_office
  '
) ORDER BY "station_id"
```

Das Ergebnis wird im Result-Tab angezeigt:



	station_id	is_ticket_office
1	1	TRUE
2	4	TRUE
3	15	TRUE

Das Ergebnis im Cyper-Tab zum Vergleich.



Für nähere Studien kann die offizielle SAP HANA GRAPH Reference verwendet werden.

https://help.sap.com/doc/21574acf46fe45a8ae9def213f2c4d9e/2.0.06/en-US/SAP_HANA_Graph_Reference_en.pdf